

Grundlagen der C-Programmierung

Dr. Henrik Brosenne
Georg-August Universität Göttingen
Institut für Informatik

Wintersemester 2024/25

Standarddatentypen, Konstanten und Variablen

Interne Darstellungen

Wertebereiche

Standardtypen

Symbolische Konstanten

Namen

Variablen

Konventionen

Binärzahlen

Heutige Computer verfügen über Schaltelemente, die zwei stabile Zustände annehmen können.

Identifiziert man diese beiden Zustände mit den Zahlen 0 und 1, so kann man **Binärzahlen**, d.h. Zahlen im Stellenwertsystem zur Basis 2, in einem Rechner speichern und verarbeiten.

Dezimalzahlen

Die **Dezimalzahlen**, mit denen zu rechnen wir gewohnt sind, lassen sich problemlos und eindeutig auch als Binärzahlen darstellen.

Beispiel

$$\begin{aligned} 109_d &= 1 \cdot 10^2 + 0 \cdot 10^1 + 9 \cdot 10^0 \\ &= 64 + 32 + 8 + 4 + 1 \\ &= 1 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 \\ &= 1101101_b \end{aligned}$$

Die dreistellige Dezimalzahl 109 ergibt eine siebenstellige Binärzahl.

Die dreistellige Dezimalzahl 999 ergibt eine zehnstellige Binärzahl (1111100111).

Oktal- und Hexadezimalzahlen

Zahlen mit vielen Ziffern sind nur sehr schlecht zu lesen, zumal wenn sie wie Binärzahlen nur aus Nullen und Einsen bestehen.

Im Zusammenhang mit Rechnern sind andere Darstellungen üblich.

- **Oktalzahlen** (Basis 8; Ziffern: 0, ..., 7)
- **Hexadezimalzahlen** (Basis 16; Ziffern: 0, ..., 9, A, ..., F).

Die Umwandlung zwischen Binär- und Oktaldarstellung bzw. zwischen Binär- und Hexadezimaldarstellung ist besonders einfach.

Standarddatentypen, Konstanten und Variablen

Interne Darstellungen

Wertebereiche

Standardtypen

Symbolische Konstanten

Namen

Variablen

Konventionen

Wertebereiche festgelegt (1/2)

Wie sollen Wertebereiche festgelegt werden?

Die arithmetischen Einheiten unterschiedliche Prozessorfamilie und sogar unterschiedliche Prozessorgenerationen rechnen mit Zahlen in unterschiedlichen Größenordnungen (Anzahl der Bits).

Ein generelles Problem ist, dass auf der einen Seite der Wertebereich eines Zahlentyps direkt von der Anzahl der Bits abhängt, die für ihn verwendet werden, dass auf der anderen Seite C aber unabhängig von bestimmter Hardware definiert werden sollte.

Wertebereiche festgelegt (1/2)

Man die Zahlen $00000000_b = 0_d$ bis $11111111_b = 255_d$ in einem Byte speichern, ein für die Praxis viel zu kleiner Wertebereich für natürliche Zahlen.

Zwei Bytes ergäben den Wertebereich 0 bis 65535, auch noch nicht übermäßig viel.

Will man neben den positiven auch negative Zahlen darstellen, so benötigt man ein Bit für das Vorzeichen.

Man erhält dann für Bytes den Wertebereich -127 (bzw. -128) bis $+127$.

Und für zwei Bytes den Wertebereich -32767 (bzw. -32768) bis $+32767$.

Man hätte nun bei der Definition von C sagen können: In den Programmen können ganzzahlige Werte mit 1, 2 oder 4 Bytes verarbeitet werden. Das wäre aber keine zweckmäßige Definition gewesen, da sie eine optimale Implementation auf vielen Rechnern verhindert hätte.

Mindestschranken

Der ANSI-Standard für C schreibt keine festen Wertebereiche, sondern nur **Mindestschranken** für die Wertebereiche vor.

Weiterhin wird festgelegt, dass für die verschiedenen Typen bestimmte **benannte Konstanten** in bestimmten Standard-Headerdateien verfügbar sein müssen, die konkreten, implementationsspezifischen Werte wiedergeben.

Diese Headerdateien sind

- `limits.h` für die Wertebereiche der ganzzahligen Typen,
- `float.h` für die Wertebereiche der Gleitkommatypen.

Standarddatentypen, Konstanten und Variablen

Interne Darstellungen

Wertebereiche

Standardtypen

Ganzzahlige Typen

Der Typ char

Zeichenkettenkonstanten

Gleitkommatypen

Symbolische Konstanten

Namen

Variablen

Konventionen

Standardtypen

Die Darstellung von ganzen Zahlen reicht nicht aus. Auch Zeichen und reelle Zahlen sollen von Programmen verarbeitet werden.

Die **Standardtypen** der Programmiersprache C tragen dem Rechnung, das sind Vorschriften, Bitfolgen bestimmter Länge in bestimmter Weise zu interpretieren.

Beschreibung ganzer Zahlen

Der Typ für die Beschreibung ganzer Zahlen (engl. *integer*) wird in C mit dem Schlüsselwort `int` bezeichnet.

Genau genommen bezeichnet `int` eine **Gruppe von Typen**.

- `short int` (kurz `short`) mit möglicherweise kleinerem Wertebereich.
- `long int` (kurz `long`) mit möglicherweise größerem Wertebereich.
- `long long int` (kurz `long long`) mit möglicherweise größerem Wertebereich.

Man hat für alle drei Wertebereiche die Wahl zwischen

- vorzeichenbehafteten Zahlen, Schlüsselwort `signed`.
- vorzeichenlosen Zahlen, Schlüsselwort `unsigned`.

Gibt man keines der beiden Schlüsselworte an, werden die Zahlen als vorzeichenbehaftet betrachtet.

Ganzzahl-Typen und Mindestschranken (1/2)

Typ	Minimum	Maximum
signed char	≤ -127	≥ 127 $= 2^7 - 1$
unsigned char	$= 0$	≥ 255 $= 2^8 - 1$
signed short int signed short short int short	≤ -32767	≥ 32767 $= 2^{15} - 1$
unsigned short int unsigned short	$= 0$	≥ 65535 $= 2^{16} - 1$
signed int signed int	≤ -32767	≥ 32767 $= 2^{15} - 1$
unsigned int	$= 0$	≥ 65535 $= 2^{16} - 1$

Ganzzahl-Typen und Mindestschranken (2/2)

Typ	Minimum	Maximum
signed long int signed long long int long	≤ -2147483647	≥ 2147483647 $= 2^{31} - 1$
unsigned long int unsigned long	$= 0$	≥ 4294967295 $= 2^{32} - 1$
signed long long int signed long long long long int long long	$\leq -9223372036854775807$	≥ 9223372036854775807 $= 2^{63} - 1$
unsigned long long int unsigned long long	$= 0$	$\geq 18446744073709551615$ $= 2^{64} - 1$

Schranken–Konstanten

Die im Standard festgelegten Namen der Schranken–Konstanten in `limits.h`.

short–Typen

`SHRT_MIN`, `SHRT_MAX`, `USHRT_MAX`

int–Typen

`INT_MIN`, `INT_MAX`, `UINT_MAX`

long–Typen

`LONG_MIN`, `LONG_MAX`, `ULONG_MAX`

long long–Typen

`LLONG_MIN`, `LLONG_MAX`, `ULLONG_MAX`

Ganzzahligen Konstanten

Ganzzahligen Konstanten im Quellcode kann man dezimal, oktal oder hexadezimal schreiben.

- Dezimale Konstanten beginnen mit einer Ziffer ungleich 0.
- Oktale Konstanten beginnen mit einer 0.
- Hexadezimale Konstanten beginnen mit 0x oder 0X;
dieser Kennung muss mindestens eine hexadezimale Ziffer folgen.

Beispiel

109	dezimal
0155	oktal
0x6D	hexadezimal
0X6D	hexadezimal

Der Typ ganzzahliger Konstanten

Eine dezimal geschriebene Konstante erhält den ersten möglichen Typ aus `int`, `long` und `long long`.

Eine oktal oder hexadezimal geschriebene Konstante erhält den ersten möglichen Typ aus `int`, `unsigned int`, `long`, `unsigned long`, `long long` und `unsigned long long`.

Der Programmierer kann den Typ einer ganzzahligen Konstanten explizit festlegen, indem er einer Konstanten Buchstaben nachstellt.

- `u` oder `U` für `unsigned`-Typen.
- `l` oder `L` für `long`-Typen.
- `ll` oder `LL` für `long long`-Typen.

Die dezimale Darstellung von 109_d im Typ `unsigned long` ist

`109lu`

Negative Konstanten

Negative Konstanten kennt C nicht!

Vorzeichen werden stets als einstellige (unäre) Operatoren interpretiert.

Bei Konstanten mit negativem Vorzeichenoperator handelt es sich jedoch um einen **konstanten Ausdruck**, deshalb werden diese trotzdem vom Compiler und nicht zur Laufzeit berechnet. Aber darauf werden wir erst später zurückkommen.

Zeichen

Zeichen (engl. *character*) werden von C als Teilmenge der ganzen Zahlen behandelt; den Typ bezeichnet das Schlüsselwort **char**.

Die verfügbaren Zeichen werden durchnummeriert, beginnend bei 0. Im Rechner werden die Zeichen dargestellt, indem diese **Ordnungszahlen** gespeichert werden.

Letztlich geht jede Programmiersprache so vor, nur ist in der Regel die Koppelung von Zahlen und Zeichen nicht so eng wie bei C.

Da die Implementation von C von der Hardware unabhängig sein soll, ergeben sich Probleme aus der Tatsache, dass unterschiedlichen Rechnerfamilien mit unterschiedlichen Zeichensätzen arbeiten.

Vorgeschriebene Zeichen

Der Standard schreibt vor, dass bestimmte Zeichen vorhanden sein müssen.

- die 26 Großbuchstaben des (englischen) Alphabets,
- die 26 Kleinbuchstaben des (englischen) Alphabets,
- die 10 (Dezimal-)Ziffern,
- 30 druckbare Sonderzeichen wie Leerzeichen, Plus und Minus, Punkt, Komma, Klammern, usw., und
- 7 Steuerzeichen zur Steuerung von Ein-/Ausgabegeräten.

Weitestgehend entsprechen diese Anforderungen des Standard der Realität des **ASCII-Code** (*American Standard Code for Information Interchange*), der insgesamt 128 Zeichen umfasst.

- Die Zeichen mit den Ordnungszahlen 0 bis 31 sowie das Zeichen mit der Ordnungszahl 127 sind Steuerzeichen.
- Die Zeichen mit den Ordnungszahlen 32 bis 126 sind druckbar.

Typ `char`

Durch die Zeichenkonstanten hat man in C jederzeit Zugriff auf die Ordnungszahl eines Zeichens, ohne den Wert selbst kennen zu müssen.

Zur internen Speicherung von Zeichen wird ein Datentyp verwendet, der über mindestens 8 Bits verfügen muss.

Obwohl ein Vorzeichen bei Zeichen nicht plausibel erscheint, unterscheidet man zwischen **signed** und **unsigned**, damit der Typ `char` kompatibel zu den `int`-Typen bleibt, deren Teilmenge er ist.

Die implementationsspezifischen Werte enthält `limits.h` in Form der Konstanten

- `CHAR_MIN` und `CHAR_MAX` für den Typ `char` ohne Zusatz,
- `SCHAR_MIN` und `SCHAR_MAX` für den Typ `char` mit dem Zusatz **signed**,
- `UCHAR_MAX` für den Typ `char` mit dem Zusatz **unsigned**.

Man sieht dass der Typ `char` ohne Zusatz, anders als die `int`-Typen, je nach Implementation vorzeichenlos oder vorzeichenbehaftet sein kann.

Zeichenkonstanten

Zeichenkonstanten sind einzelne Zeichen, eingeschlossen in Apostrophe, etwa

'w'

Eine Zeichenkonstante repräsentiert die Ordnungszahl ihres Zeichens, welche das ist, hängt vom Zeichensatz ab, den der Rechner verwendet.

Wichtig. Die Zeichenkonstante '0' repräsentiert in der Regel **nicht** die Ordnungszahl 0, die Zeichenkonstante '1' **nicht** die Ordnungszahl 1, usw.

Vergleiche ASCII-Code, die Ordnungszahlen der druckbaren Zeichen beginnen in ihm erst bei 32.

Einige Zeichen können nicht ohne weiteres angegeben werden.

- Einige druckbare Zeichen haben im Quellcode bestimmte Bedeutung, z.B. der Apostroph.
- Die Steuerzeichen für Ein-/Ausgabegeräte, die C definiert, stehen auf der Tastatur nicht zur Verfügung.

Escape-Sequenzen

Zeichen	Beschreibung
\a	Piepen (wird nicht von allen Ausgabegeräten unterstützt)
\b	Backspace
\f	Seitenvorschub (wird nur von Druckern beachtet)
\n	Zeilenvorschub
\r	Positionierung auf Zeilenanfang
\t	Horizontaler Tabulator
\v	Vertikaler Tabulator (wird nur von Druckern beachtet)
\'	Apostroph, der nicht zur Begrenzung einer Zeichenkonstante dient
\"	Anführungszeichen, die nicht zur Begrenzung einer Stringkonstante dienen
\\	Backslash
\?	Fragezeichen. Nur relevant, wenn man zwei unmittelbar aufeinander folgende Fragezeichen in einer Zeichenkettenkonstanten verwenden möchte. Hintergrund ist die selten benötigte Möglichkeit, Sonderzeichen durch eine Kombination aus zwei Fragezeichen und einem weiteren Zeichen darzustellen („Trigraphen“).

Zeichenkettenkonstanten

Eine Zeichenkettenkonstante (engl. *String*) ist eine Folge von beliebig vielen Zeichen, die in Anführungszeichen eingeschlossen ist.

Strings, die nur durch *white spaces* getrennt sind, werden vom Präprozessor zu einem einzigen String zusammengefügt.

```
printf("Dieser String ist zu lang, um ihn in einer "  
      "einzelnen Zeile unterzubringen.\n");
```

Der Backslash \ ist in einer Zeichenkettenkonstante nur in Verbindung mit einer Escape-Sequenz erlaubt.

Zeilenumbrüche sind nicht erlaubt. Soll ein Zeilenumbruch in der Ausgabe erfolgen, so ist die Escape-Sequenz \n zu verwenden.

Da Anführungszeichen zur Begrenzung von Strings dienen, können diese ebenfalls nur als Escape-Sequenz \" innerhalb eines Strings auftauchen.

Achtung. 'x' bedeutet nicht dasselbe wie "x".

Gleitkommazahlen

Zum numerischen Rechnen benötigt man **Gleitkommazahlen**.

C stellt Gleitkommazahlen mit den folgenden Typen zur Verfügung.

- `float`
- `double`
- `long double`

Alle drei Typen müssen mindestens den Wertebereich

$$[-10^{+37}, -10^{-37}] \cup \{0\} \cup [+10^{-37}, +10^{+37}]$$

besitzen.

Zur Beschreibung von Gleitkommazahlen reicht die Angabe des Wertebereichs allerdings nicht aus; ebenso benötigt man die Angabe der **Dichte** der Zahlen.

Die Dichte ist der Abstand der Zahl 1 von der nächstgelegenen größeren Zahl.

Der Standard verlangt die minimalen Dichten

- 10^{-5} beim Typ `float`.
- 10^{-9} bei den Typen `double` und `long double`.

Das entspricht einer Genauigkeit von 6 bzw. 10 dezimal-Stellen.

Wertebereiche der Gleitkommatypen

Die Dichte bezieht sich auf Dezimalstellen, aber Gleitkommazahlen werden intern binär verarbeitet,

Bei der Umwandlung von der einen in die andere Form treten zwangsläufig Rundungsfehler auf.

Die Mindestanzahl an Dezimalstellen, die bei Umwandlung in Binärdarstellung und Rückumwandlung erhalten bleibt, wird durch die Anzahl der **signifikanten Stellen** angegeben, auch dafür macht der Standard eine Vorgabe.

Die konkreten Werte für die Wertebereiche der Gleitkommatypen findet man in der Datei `float.h`.

- Wertebereich. `FLT_MAX`, `DBL_MAX`, `LDBL_MAX`
- Dichte. `FLT_EPSILON`, `DBL_EPSILON`, `LDBL_EPSILON`
- Anzahl der signifikanten Stellen. `FLT_DIG`, `DBL_DIG`, `LDBL_DIG`.

Darstellung von Gleitkommazahlen

Im Quelltext hat man prinzipiell zwei Möglichkeiten der Darstellung.

- Dezimalpunkt. Ein `.` vor, inmitten oder hinter der Ziffernfolge.
- Exponententeil. Ziffernfolge gefolgt von `e` oder `E`, danach der Exponenten zur Basis 10 als ganze Zahl mit oder ohne Vorzeichen.

Man kann beide Möglichkeiten auch kombinieren.

Beispiele

<code>.1</code>	<code>0.1</code>	<code>1e-1</code>	<code>1.0E-1</code>
<code>1.</code>	<code>1.0</code>	<code>1e0</code>	<code>1.0E+0</code>
<code>10.</code>	<code>10.0</code>	<code>1e1</code>	<code>1.0E+1</code>

Ohne weiteres besitzen Gleitkommakonstanten den Typ `double`.

Festlegung des Typs durch nachgestellten Kennbuchstaben

- `f` oder `F` für `float`,
- `l` oder `L` für `long double`.

Standarddatentypen, Konstanten und Variablen

Interne Darstellungen

Wertebereiche

Standardtypen

Symbolische Konstanten

Namen

Variablen

Konventionen

Symbolische Konstanten

Symbolische Konstanten sind für den Programmierer wichtig, um „magische Zahlen“ zu vermeiden.

Beispiel

Ein Programm zur Berechnung von Verkaufspreisen.

Steht an allen Stellen, an denen Mehrwertsteuer berechnet wird, explizit die Konstante 16 bzw. 1.16, so gab es bei der Erhöhung der Mehrwertsteuer am 01.01.2007 viel Arbeit für die Programmierer.

Überall mussten diese Konstanten ersetzt werden. Ein besonderes Problem dabei ist, dass für jede 16 bzw. 1.16 separat entschieden werden musste, ob tatsächlich die Mehrwertsteuer oder eine andere Größe bezeichnete wird, die nicht verändert werden sollte.

#define

Mit Hilfe der Präprozessor-Direktive

```
#define name ersatztext
```

kann man Konstanten sinnvolle Namen geben, etwa

```
#define MWST 19.0
```

In allen nachfolgenden Zeilen wird an allen Stellen, an denen *name* nicht innerhalb eines String oder als Teil eines anderen Namens steht, *name* durch *ersatztext* ersetzt.

Für *ersatztext* können beliebige Werte angegeben werden.

Es ist üblich für die Namen von symbolischen Konstanten ausschließlich Großbuchstaben zu verwenden, auch wenn der Standard das nicht vorschreibt.

Standarddatentypen, Konstanten und Variablen

Interne Darstellungen

Wertebereiche

Standardtypen

Symbolische Konstanten

Namen

Variablen

Konventionen

Namen

Um verschiedene Speicherplätze und andere Objekte, die in C-Programmen eine Rolle spielen, ansprechen zu können, muss man sie mit einem **Namen** oder **Bezeichner** versehen.

Zur Bildung von Namen stehen folgende Zeichen zur Verfügung:

- uneingeschränkt die 52 Buchstaben, wobei zwischen Groß- und Kleinbuchstaben strikt unterschieden wird,
- die 10 Ziffern, wobei das erste Zeichen eines Namen keine Ziffer sein darf, um eine Unterscheidung von numerischen Konstanten zu erlauben,
- das Unterstreichungszeichen (_). Dieses Zeichen darf auch am Anfang eines Namen stehen; sollte an dieser Stelle aber für Bibliotheksfunktionen reserviert bleiben.

Länge von Namen

Namen dürfen zwar beliebig lang sein, jedoch schreibt der Standard vor, dass die ersten 63 Zeichen **signifikant** sein müssen.

Ein Compiler darf also Namen als gleich interpretieren, die sich erst im 64. oder einem späteren Zeichen unterscheiden.

Die Minimalschranke für die Anzahl signifikanter Zeichen von Namen, die über eine Quelldatei hinaus verwendet werden sollen, liegt nur bei 31.

Schlüsselwörter

Bestimmte Zeichenfolgen, die **Schlüsselwörter**, sind zwar wie Namen aufgebaut, dürfen aber nur mit ihrer festgelegten Bedeutung verwendet werden.

<code>auto</code>	<code>extern</code>	<code>short</code>	<code>while</code>
<code>break</code>	<code>float</code>	<code>signed</code>	<code>_Alignas</code>
<code>case</code>	<code>for</code>	<code>sizeof</code>	<code>_Alignof</code>
<code>char</code>	<code>goto</code>	<code>static</code>	<code>_Atomic</code>
<code>const</code>	<code>if</code>	<code>struct</code>	<code>_Bool</code>
<code>continue</code>	<code>inline</code>	<code>switch</code>	<code>_Complex</code>
<code>default</code>	<code>int</code>	<code>typedef</code>	<code>_Generic</code>
<code>do</code>	<code>long</code>	<code>union</code>	<code>_Imaginary</code>
<code>double</code>	<code>register</code>	<code>unsigned</code>	<code>_Noreturn</code>
<code>else</code>	<code>restrict</code>	<code>void</code>	<code>_Static_assert</code>
<code>enum</code>	<code>return</code>	<code>volatile</code>	<code>_Thread_local</code>

Da auch bei den Schlüsselwörtern zwischen Groß- und Kleinbuchstaben unterschieden wird, könnte man zum Beispiel zwar `Int` oder `INT` als Namen verwenden, nicht jedoch `int`.

Standarddatentypen, Konstanten und Variablen

Interne Darstellungen

Wertebereiche

Standardtypen

Symbolische Konstanten

Namen

Variablen

Konventionen

Variablen

Konstanten verändern ihren Wert zur Laufzeit eines Programms nicht. Sie besitzen außerdem nur ihren Wert und Typ, nicht jedoch einen Speicherplatz.

Variablen besitzen einen Typ, einen Speicherplatz und in der Regel auch einen Wert, nämlich den, der an diesem Speicherplatz steht.

Solange der Wert einer Variablen nicht gesetzt wurde, braucht sie keinen wohldefinierten Wert zu besitzen.

In C müssen alle Variablen deklariert werden, bevor sie verwendet werden können.

Deklaration von Variablen (1/3)

Die Deklaration von Variablen hat die Form

`[speicherklasse] typ namenliste ;`

- Die Angabe eines Speicherklassen-Attributs ist optional.
- *typ* ist eines der Schlüsselwörter oder eine der Schlüsselwort-Kombinationen, die wir bereits kennengelernt haben. Auf weitere Möglichkeiten werden wir später zurückkommen.
- Die *namenliste* enthält, durch je ein Komma voneinander getrennt, die Namen der zu deklarierenden Variablen.
- Die Deklaration wird durch ein Semikolon abgeschlossen.

Beispiele

```
int i;  
double a, b, c;
```

Deklaration von Variablen (2/3)

Die Deklaration einer Variablen hat zwei Funktionen.

- Der Compiler erhält die Größe des Speicherplatzes, der für die Variablen bereitgestellt werden muss.
- Der Compiler erhält die Vorschrift, wie die Bitfolge, die sich an dem Speicherplatz einer Variablen befindet, zu interpretieren ist (als ganze Zahl mit oder ohne Vorzeichen, als Gleitkommazahl, usw.).

Die Deklaration einer Variablen bewirkt im Allgemeinen ohne weiteres nicht gleichzeitig einen wohldefinierten Anfangswert für die Variable.

Ein Anfangswert kann angegeben werden (Initialisierung), indem man die Deklaration um ein Gleichheitszeichen und den gewünschten Wert erweitert

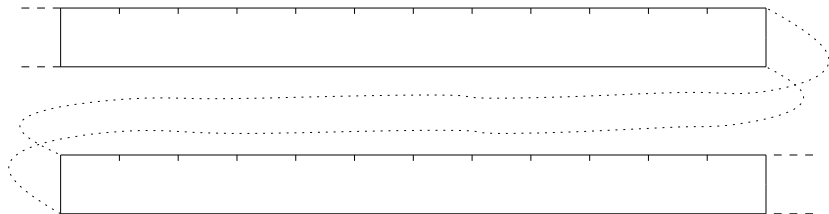
Beispiel

```
double x = 2.5e-7;  
int i, summe = 0, quotient = 1, j;
```

Anfangswerte müssen immer individuell zugewiesen werden; eine Schreibweise, um mehreren Variablen denselben Anfangswert zuzuweisen, gibt es nicht.

Deklaration von Variablen (3/3)

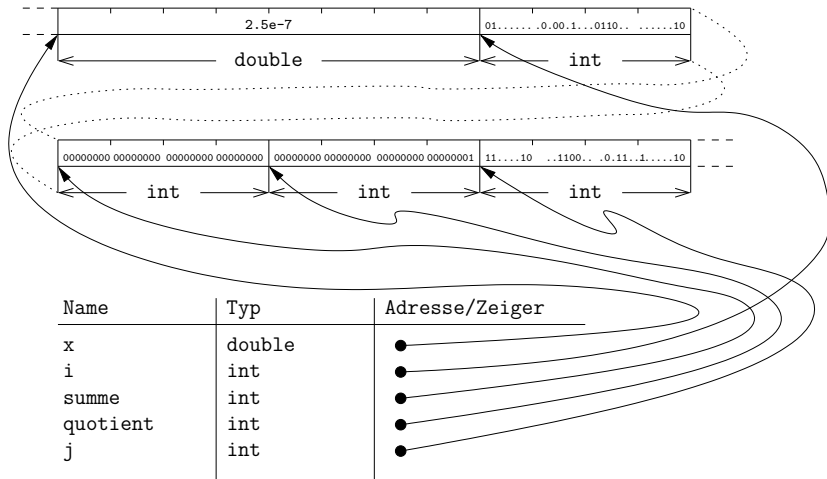
```
double x = 2.5e-7;
int i, somme = 0, quotient = 1, j;
```



Name	Typ	Adresse/Zeiger

Deklaration von Variablen (3/3)

```
double x = 2.5e-7;  
int i, summe = 0, quotient = 1, j;
```



Standarddatentypen, Konstanten und Variablen

Interne Darstellungen

Wertebereiche

Standardtypen

Symbolische Konstanten

Namen

Variablen

Konventionen

Konventionen für Namen (1/2)

Um die Verständlichkeit von Quelltexten zu erhöhen, sollte man bei der Namensvergabe gewissen Konventionen folgen.

Es ist in C üblich, für die Namen von

- Variablen nur Kleinbuchstaben,
- benannte Konstanten nur Großbuchstaben

zu verwenden.

In der Regel sollte man die Namen von Variablen und Konstanten so wählen, dass sie etwas über die Bedeutung ihres Wertes aussagen.

In der Regel findet man in gut formulierten Quelltexten nur selten Variablennamen, die nur aus ein oder zwei Buchstaben bestehen. Vernünftige Namen sind häufig 4 bis 6 Zeichen lang, oft auch länger.

Konventionen für Namen (2/2)

Sinnvoll sind sehr kurze Bezeichner oft nur in folgenden Fällen.

- Bei der Realisierung mathematischer Formeln sollte man die Bezeichnungen aus den Formeln übernehmen, soweit das möglich ist.
- Hochgradig lokale Variablen, etwa Zählvariablen in Schleifen, haben häufig keine benennbare Bedeutung.

i, j, k	(Lauf-)Indizes
m, n	Anzahlen
c, ch	Zeichen
s	Zeichenketten („Strings“)
p, z	Zeiger

Hier sieht man, dass die Länge eines Namens ist nicht alles ist. Für den Geübten hat der Name `i` ebenso viel (oder wenig) Aussagekraft wie der Name `laufvariable`.

Und noch etwas sollte man beachten.

- Zu ähnliche Namen verleiten zu Schreib- und Lesefehlern.
- Die Ziffer 1 und die Buchstaben I und l verwechselt man besonders leicht, ebenso die Ziffer 0 und den Buchstaben O.